

```
import pandas as pd
import numpy as np

from sklearn.model_selection import train_test_split
from sklearn.tree import DecisionTreeRegressor
from sklearn.preprocessing import LabelEncoder

from sklearn.metrics import mean_absolute_error, mean_squared_error, r2_score
```

```
df = pd.read_csv('/content/MBA.csv')
df.head()
```

	application_id	gender	international	gpa	major	race	gmat	work_exp	work_industry	admission	
0	1	Female	False	3.30	Business	Asian	620.0	3.0	Financial Services	Admit	
1	2	Male	False	3.28	Humanities	Black	680.0	5.0	Investment Management	NaN	
2	3	Female	True	3.30	Business	NaN	710.0	5.0	Technology	Admit	
3	4	Male	False	3.47	STEM	Black	690.0	6.0	Technology	NaN	
4	5	Male	False	3.35	STEM	Hispanic	590.0	5.0	Consulting	NaN	

Next steps:

[Generate code with df](#)

[New interactive sheet](#)

```
df.isnull().sum()
```

	0
application_id	0
gender	0
international	0
gpa	0
major	0
race	1842
gmat	0
work_exp	0
work_industry	0
admission	5194

dtype: int64

```
num_cols = df.select_dtypes(include=np.number).columns
```

```
for col in num_cols:
    df[col].fillna(df[col].median(), inplace=True)
```

/tmp/ipykernel_529/3551297251.py:4: FutureWarning: A value is trying to be set on a copy of a DataFrame or Series through chained assignment. The behavior will change in pandas 3.0. This inplace method will never work because the intermediate object on which we are setting values is a copy.

For example, when doing 'df[col].method(value, inplace=True)', try using 'df.method({col: value}, inplace=True)' or df[col] = df[col].method(value, inplace=True)

```
df[col].fillna(df[col].median(), inplace=True)
```

```
cat_cols = df.select_dtypes(include='object').columns
```

```
for col in cat_cols:
    df[col].fillna(df[col].mode()[0], inplace=True)
```

/tmp/ipykernel_529/2422164989.py:4: FutureWarning: A value is trying to be set on a copy of a DataFrame or Series through chained assignment. The behavior will change in pandas 3.0. This inplace method will never work because the intermediate object on which we are setting values is a copy.

For example, when doing 'df[col].method(value, inplace=True)', try using 'df.method({col: value}, inplace=True)' or df[col] = df[col].method(value, inplace=True)

```
df[col].fillna(df[col].mode()[0], inplace=True)
```

```
df.isnull().sum()
```

```
      0
application_id  0
gender         0
international  0
gpa           0
major         0
race          0
gmat          0
work_exp      0
work_industry  0
admission     0

dtype: int64
```

```
le = LabelEncoder()
```

```
for col in df.select_dtypes(include='object').columns:
    df[col] = le.fit_transform(df[col])
```

```
x = df.drop('gpa', axis=1)
y = df['gpa']
```

```
x_train, x_test, y_train, y_test = train_test_split(
    x, y, test_size=0.2, random_state=42)
```

```
depth_values = [2,4,6,8]
```

```
criteria = ['squared_error', 'absolute_error']
```

```
results = []
```

```

for c in criteria:
    for depth in depth_values:

        model = DecisionTreeRegressor(
            criterion=c,
            max_depth=depth,
            random_state=42)

        model.fit(x_train,y_train)

        y_pred = model.predict(x_test)

        mae = mean_absolute_error(y_test,y_pred)
        mse = mean_squared_error(y_test,y_pred)
        rmse = np.sqrt(mse)
        r2 = r2_score(y_test,y_pred)

        print("\n=====")
        print("Criterion:",c)
        print("Max Depth:",depth)

        print("MAE:",mae)
        print("MSE:",mse)
        print("RMSE:",rmse)
        print("R2 Score:",r2)

        results.append([c,depth,mae,mse,rmse,r2])

```

```

RMSE: 0.12713568264247835
R2 Score: 0.3134686328641362

```

```

=====
Criterion: squared_error
Max Depth: 4
MAE: 0.1006175982316143
MSE: 0.015799583668917854
RMSE: 0.12569639481273062
R2 Score: 0.3289249240996214
=====

```

```

=====
Criterion: squared_error
Max Depth: 8
MAE: 0.1074436950629798
MSE: 0.018202944834250827
RMSE: 0.13491828947274281
R2 Score: 0.2268440205619705

=====
Criterion: absolute_error
Max Depth: 2
MAE: 0.10242937853107344
MSE: 0.01650225988700565
RMSE: 0.12846112208370924
R2 Score: 0.2990792961218206

=====
Criterion: absolute_error
Max Depth: 4
MAE: 0.100863599677159
MSE: 0.015954237288135595
RMSE: 0.12631008387351975
R2 Score: 0.32235613143839537

=====
Criterion: absolute_error
Max Depth: 6
MAE: 0.1031557707828894
MSE: 0.016626311541565773
RMSE: 0.12894305542201864
R2 Score: 0.2938102982011175

=====
Criterion: absolute_error
Max Depth: 8
MAE: 0.10690879741727198
MSE: 0.01790815173527038
RMSE: 0.13382134260001421
R2 Score: 0.23936512905563245

```

```

results_df = pd.DataFrame(results,
                           columns=['Criterion', 'Depth', 'MAE', 'MSE', 'RMSE', 'R2'])

results_df

```

	Criterion	Depth	MAE	MSE	RMSE	R2	
0	squared_error	2	0.101733	0.016163	0.127136	0.313469	
1	squared_error	4	0.100618	0.015800	0.125696	0.328925	
2	squared_error	6	0.102258	0.016343	0.127841	0.305830	
3	squared_error	8	0.107444	0.018203	0.134918	0.226844	
4	absolute_error	2	0.102429	0.016502	0.128461	0.299079	
5	absolute_error	4	0.100864	0.015954	0.126310	0.322356	

Next steps: [Generate code with results_df](#) [New interactive sheet](#)

```
results_df.sort_values(by='R2',ascending=False)
```

	Criterion	Depth	MAE	MSE	RMSE	R2	
1	squared_error	4	0.100618	0.015800	0.125696	0.328925	
5	absolute_error	4	0.100864	0.015954	0.126310	0.322356	
0	squared_error	2	0.101733	0.016163	0.127136	0.313469	
2	squared_error	6	0.102258	0.016343	0.127841	0.305830	
4	absolute_error	2	0.102429	0.016502	0.128461	0.299079	
6	absolute_error	6	0.103156	0.016626	0.128943	0.293810	
7	absolute_error	8	0.106909	0.017908	0.133821	0.239365	
3	squared_error	8	0.107444	0.018203	0.134918	0.226844	

Start coding or [generate](#) with AI.